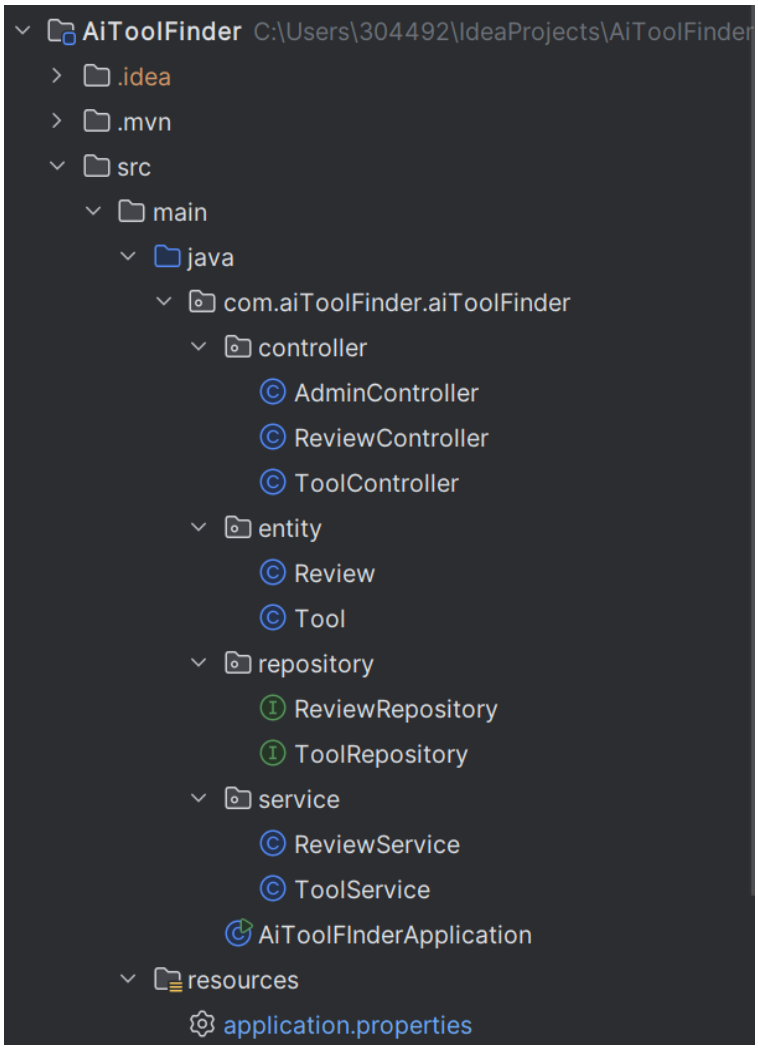
AI Tool Finder - Curated Tools Platform

Project Overview

The AI Tool Finder is a backend-only RESTful application designed to simulate a real-world system for discovering and reviewing AI tools. The platform allows users to explore AI tools, apply advanced filters, submit reviews, and see automatically computed ratings, while administrators manage tools and moderate user reviews.

This project focuses on backend engineering principles such as clean API design, business logic implementation, data persistence, filtering, aggregation, and role-based operations.

Project Structure



Admin Controller

The Admin Controller is responsible for handling all administrator-level

operations in the AI Tool Finder application. These operations include managing AI tools and moderating user reviews. To maintain system integrity and data quality, all admin functionalities are separated from public APIs and are accessible only through endpoints prefixed with /admin.

This controller follows a clean layered architecture by delegating all business logic to the service layer, ensuring that the controller remains lightweight and focused only on request handling.

Functionalities Provided

1. Add New AI Tool

The admin can add a new AI tool by providing tool details such as name, category, use case, and pricing type. This enables the system to continuously expand the list of available AI tools.

2. Update Existing AI Tool

The admin can update the details of an existing AI tool. This functionality ensures that outdated or incorrect tool information can be corrected efficiently.

3. Approve User Reviews

User reviews submitted through public APIs are initially stored with a **pending** status. The admin reviews these submissions and approves valid reviews. Only approved reviews are considered when calculating the average rating of an AI tool.

Architectural Role

The Admin Controller acts as an intermediary between the client and the service layer:

- It receives HTTP requests from admin users
- It validates and maps incoming data
- It calls appropriate service methods to perform business logic
- It returns meaningful responses to the client

This separation improves maintainability, scalability, and security of the backend system.

```java
package com.aiToolFinder.aiToolFinder.controller;

import ...

@RestController  no usages  AbhisheSumod10 +1
@RequestMapping("/admin")
public class AdminController {

    private final ToolService toolService;  3 usages
    private final ReviewService reviewService;  2 usages

    public AdminController(ToolService toolService,  no usages  AbhisheSumod10
                           ReviewService reviewService) {
        this.toolService = toolService;
        this.reviewService = reviewService;
    }

    // ADMIN: Add tool
    @PostMapping("/tools")  no usages  AbhisheSumod10
    public Tool addTool(@RequestBody Tool tool) { return toolService.addTool(tool); }

    // ADMIN: Update tool
    @PutMapping("/tools/{id}")  no usages  AbhisheSumod10
    public Tool updateTool(@PathVariable Long id,
                           @RequestBody Tool tool) {
        return toolService.updateTool(id, tool);
    }
```

```java
    // ADMIN: Approve review
    @PostMapping("/reviews/{id}/approve")  no usages  AbhisheSumod10
    public String approveReview(@PathVariable Long id) {
        reviewService.approveReview(id);
        return "Review approved";
    }
}
```

Review Controller

The Review Controller handles all user-side review submission operations in the AI Tool Finder application. It allows users to submit ratings and optional comments for AI tools. To ensure quality and prevent misuse, all submitted reviews are initially stored with a pending approval status and are later reviewed by an administrator.

This controller exposes a simple and secure API that forwards user input to the service layer for processing.

```java
package com.aiToolFinder.aiToolFinder.controller;

import ...

@RestController  no usages  AbhisheSumod10 +1
@RequestMapping("/review")
public class ReviewController {

    private final ReviewService reviewService;  2 usages

    public ReviewController(ReviewService reviewService) { this.reviewService = reviewService; }

    // USER: Submit review
    @PostMapping  no usages  AbhisheSumod10
    public String submitReview(@RequestParam Long toolId,
                               @RequestParam int rating,
                               @RequestParam(required = false) String comment) {

        reviewService.submitReview(toolId, rating, comment);
        return "Review submitted for approval";
    }
}
```

Workflow

1. User submits a review using the review endpoint.
2. The review is saved with a pending status.
3. The admin reviews the submission.
4. Once approved, the review contributes to the tool's average rating.

Architectural Role

The Review Controller acts as the entry point for user-generated reviews:

- Handles incoming HTTP requests.
- Passes validated data to the service layer.

- Keeps business logic out of the controller.
- Ensures a clean separation of concerns.

Tool Controller

The Tool Controller is responsible for handling all user-facing AI tool retrieval operations in the AI Tool Finder application. It allows users to view the list of available AI tools and apply multiple filters such as category, pricing type, and minimum rating. This enables users to easily discover AI tools that match their specific requirements.

The controller acts as a bridge between client requests and the service layer, ensuring clean separation of concerns.

```java
package com.aiToolFinder.aiToolFinder.controller;

import ...

@RestController  no usages  AbhisheSumod10 +1
@RequestMapping("/tools")
public class ToolController {

    private final ToolService toolService;  2 usages

    public ToolController(ToolService toolService) { this.toolService = toolService; }

    // USER: Get tools with filters
    @GetMapping  no usages  AbhisheSumod10
    public List<Tool> getTools(
            @RequestParam(required = false) String category,
            @RequestParam(required = false) String pricingType,
            @RequestParam(required = false) Double minRating
    ) {
        return toolService.getFilteredTools(category, pricingType, minRating);
    }
}
```

Workflow

1. User sends a request with optional filter parameters.
2. Controller forwards the parameters to Tool Service.
3. Service layer applies filtering logic and fetches matching tools.
4. Controller returns the filtered list to the user.

Architectural Role

The Tool Controller serves as the entry point for tool discovery:

- Handles HTTP GET requests.
- Passes filter criteria to the service layer.
- Returns filtered results to the client.
- Maintains a clean and scalable architecture.

Review Entity

The Review entity represents a user review for an AI tool in the AI Tool Finder application. Each review contains a user-provided rating, an optional comment, and a moderation status that determines whether the review contributes to the tool's overall rating.

This entity is mapped to a database table using JPA (Jakarta Persistence API) and is used to store, retrieve, and manage review data in the system.

```java
    public void setId(Long id) { this.id = id; }

    public Long getToolId() { return toolId; }

    public void setToolId(Long toolId) { this.toolId = toolId; }

    public int getUserRating() { return userRating; }

    public void setUserRating(int userRating) { this.userRating = userRating; }

    public String getComment() { return comment; }

    public void setComment(String comment) { this.comment = comment; }

    public String getStatus() { return status; }

    public void setStatus(String status) { this.status = status; }

    @Override    Prabu Sankar Rajini(UST
    public String toString() {
        return "Review{" +
                "id=" + id +
                ", toolId=" + toolId +
                ", userRating=" + userRating +
                ", comment='" + comment + '\'' +
                ", status='" + status + '\'' +
                '}';
    }
```

Entity Fields and Their Purpose

1.toolId

private Long toolId;

- Stores the ID of the AI tool being reviewed
- Establishes a logical relationship between a review and a tool

2. userRating

private int userRating;

- Stores the rating given by the user
- Typically ranges from 1 to 5
- Used to calculate the average rating of a tool after approval.

3. comment

private String comment;

- Stores optional textual feedback from the user
- Helps users share their experience with the tool

4.status

private String status;

- Represents the review's moderation state
- Possible values:
    - PENDING – Awaiting admin approval
    - APPROVED – Included in rating calculation
    - REJECTED – Ignored by the system

Constructors

No-Argument Constructor

public Review() {}

- Required by JPA for object creation during database operations.

Parameterized Constructor

public Review(Long id, Long toolId, int userRating, String comment, String status)

- Allows easy creation of Review objects with all fields initialized

Getters and Setters

- Provide controlled access to private fields
- Used by:
    - Service layer for business logic
    - JPA for persistence operations

toString() Method

@Override

public String toString()

- Provides a readable string representation of the Review object
- Useful for debugging and logging purposes.

Tool Entity

The Tool entity represents an AI tool in the AI Tool Finder application. It stores all the important details required to describe an AI tool and helps users discover tools based on their needs. This class is mapped to a database table using JPA (Jakarta Persistence API) and is used for persistent storage of tool information.

Entity Annotation

@Entity

- Marks this class as a JPA entity
- Automatically maps it to a database table named tool
- Each object of this class corresponds to one record in the database.

```java
package com.aiToolFinder.aiToolFinder.entity;
import jakarta.persistence.*;

@Entity  18 usages  Prabu Sankar Rajini(UST +1
public class Tool {

    @Id  4 usages
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private String name;  4 usages
    private String useCase;  4 usages
    private String category;  4 usages
    private String pricingType;  4 usages
    private double rating;  4 usages

    public Tool() {  no usages  Prabu Sankar Rajini(UST
    }

    public Tool(Long id, String name, String useCase, String category, String pricingType, double rati
        this.id = id;
        this.name = name;
        this.useCase = useCase;
        this.category = category;
        this.pricingType = pricingType;
        this.rating = rating;
    }
```

```java
public String getUseCase() { return useCase; }

public void setUseCase(String useCase) { this.useCase = useCase; }

public String getCategory() { return category; }

public void setCategory(String category) { this.category = category; }

public String getPricingType() { return pricingType; }

public void setPricingType(String pricingType) { this.pricingType = pricingType; }

public double getRating() { return rating; }

public void setRating(double rating) { this.rating = rating; }

@Override  Prabu Sankar Rajini(UST
public String toString() {
    return "Tool{" +
            "id=" + id +
            ", name='" + name + '\'' +
            ", useCase='" + useCase + '\'' +
            ", category='" + category + '\'' +
            ", pricingType='" + pricingType + '\'' +
            ", rating=" + rating +
            '}';
}
```

Review Repository

```java
package com.aiToolFinder.aiToolFinder.repository;

import ...

public interface ReviewRepository extends JpaRepository<Review,Long> {
}
```

The Review Repository is responsible for performing database operations related to the Review entity. It extends Jpa Repository, which provides built-in methods for common CRUD operations such as saving, updating, deleting, and fetching reviews from the database.

By using Jpa Repository, this repository eliminates the need to write manual SQL queries and enables seamless interaction between the application and the database.

Tool Repository

```java
package com.aiToolFinder.aiToolFinder.repository;

import ...

public interface ToolRepository extends JpaRepository<Tool,Long> {
}
```

The Tool Repository is used to perform database operations related to the Tool entity. It extends Jpa Repository, which provides ready-made methods for creating, reading, updating, and deleting AI tool records without writing SQL queries.

This repository acts as the link between the application and the database for managing AI tool data.

Review Service

```java
package com.aiToolFinder.aiToolFinder.service;

import ...

@Service  6 usages  AbhisheSumod10
public class ReviewService {

    private final ReviewRepository reviewRepository;  5 usages
    private final ToolRepository toolRepository;  3 usages

    public ReviewService(ReviewRepository reviewRepository,  no usages
                         ToolRepository toolRepository) {
        this.reviewRepository = reviewRepository;
        this.toolRepository = toolRepository;
    }

    // USER: Submit review
    public void submitReview(Long toolId, int rating, String comment)

        Review review = new Review();
        review.setToolId(toolId);
        review.setUserRating(rating);
        review.setComment(comment);
        review.setStatus("PENDING");

        reviewRepository.save(review);
    }
```

```java
    public void approveReview(Long reviewId) {  1 usage  AbhisheSumod10 *

        Review review = reviewRepository.findById(reviewId)
                .orElseThrow(() -> new RuntimeException("Review not found"));

        review.setStatus("APPROVED");
        reviewRepository.save(review);
        updateToolRating(review.getToolId());
    }

    // Calculate average rating
    private void updateToolRating(Long toolId) {  1 usage  AbhisheSumod10 *
        List<Review> reviews = reviewRepository.findAll();
        double avgRating = reviews.stream()  Stream<Review>
                .filter( Review r -> r.getToolId().equals(toolId))
                .filter( Review r -> r.getStatus().equals("APPROVED"))
                .mapToInt(Review::getUserRating)  IntStream
                .average()  OptionalDouble
                .orElse( v: 0.0);

        Tool tool = toolRepository.findById(toolId)
                .orElseThrow(() -> new RuntimeException("Tool not found"));
        tool.setRating(avgRating);
        toolRepository.save(tool);
    }
}
```

The Review Service handles the business logic related to user reviews.

- submitReview → Saves a new review with status PENDING.
- approveReview → Admin approves a review, changes its status to APPROVED, and triggers rating update.
- updateToolRating → Calculates the average rating of a tool using only approved reviews and updates the tool's rating.

It interacts with Review Repository and Tool Repository to manage data in the database.

Tool Service

```java
import ...

@Service  6 usages  AbhisheSumod10 +1
public class ToolService {

    private final ToolRepository toolRepository;  5 usages

    public ToolService(ToolRepository toolRepository) { this.toolRepository = toolRepository; }

    // USER: Fetch & filter tools
    public List<Tool> getFilteredTools(String category, String pricingType, Double minRating) {  1 usage

        return toolRepository.findAll().stream()
                .filter( Tool t -> category == null || t.getCategory().equalsIgnoreCase(category))
                .filter( Tool t -> pricingType == null || t.getPricingType().equalsIgnoreCase(pricingType)
                .filter( Tool t -> minRating == null || t.getRating() >= minRating)
                .collect(Collectors.toList());
    }

    // ADMIN: Add new tool
    public Tool addTool(Tool tool) { return toolRepository.save(tool); }

    // ADMIN: Update existing tool
    public Tool updateTool(Long id, Tool updatedTool) {  1 usage  AbhisheSumod10

        Tool existingTool = toolRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Tool not found"));
```

```java
    // ADMIN: Update existing tool
    public Tool updateTool(Long id, Tool updatedTool) {  1 usage  AbhisheSumod10

        Tool existingTool = toolRepository.findById(id)
                .orElseThrow(() -> new RuntimeException("Tool not found"));

        if (updatedTool.getName() != null)
            existingTool.setName(updatedTool.getName());

        if (updatedTool.getUseCase() != null)
            existingTool.setUseCase(updatedTool.getUseCase());

        if (updatedTool.getCategory() != null)
            existingTool.setCategory(updatedTool.getCategory());

        if (updatedTool.getPricingType() != null)
            existingTool.setPricingType(updatedTool.getPricingType());

        return toolRepository.save(existingTool);
    }
}
```

Tool Service

The Tool Service contains the business logic for managing AI tools in the application.

- getFilteredTools → Fetches all tools and applies optional filters: category, pricing type, and minimum rating. Used by users to discover tools.
- addTool → Saves a new AI tool to the database. Used by admins.
- updateTool → Updates an existing tool's details if provided. Used by admins to maintain accurate tool information.

It interacts with Tool Repository to perform database operations.

```properties
# application.properties
spring.datasource.url=jdbc:mysql://localhost:3306/aitoolfinder?allowPublicKeyRetrieval=true&useSSL=false
spring.datasource.username=swathi
spring.datasource.password=swathi1234
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# JPA/Hibernate (Hibernate 7 -> unified MySQL Dialect)
spring.jpa.database-platform=org.hibernate.dialect.MySQLDialect
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
spring.jpa.properties.hibernate.format_sql=true
```

Security Configuration

```java
package com.aiToolFinder.aiToolFinder.config;

import ...

@Configuration  no usages  VikramS03
public class SecurityConfig {

    @Bean  no usages  VikramS03
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {

        http
                .csrf( CsrfConfigurer<HttpSecurity> csrf -> csrf.disable())
                .authorizeHttpRequests( AuthorizationManagerRequestMat... auth -> auth
                        .requestMatchers( ...patterns: "/admin/**").hasRole("ADMIN")
                        .requestMatchers( ...patterns: "/tools/**", "/reviews/**").hasAnyRole( ...roles: "USER",
                        .anyRequest().authenticated()
                )
                .httpBasic(Customizer.withDefaults()); //

        return http.build();
    }

    @Bean  no usages  VikramS03
    public UserDetailsService users() {
        return new InMemoryUserDetailsManager(
                User.withUsername("admin")
                        .password("{noop}admin123")
                        .roles("ADMIN")
```

```java
    @Bean  no usages  VikramS03
    public UserDetailsService users() {
        return new InMemoryUserDetailsManager(
                User.withUsername("admin")
                        .password("{noop}admin123")
                        .roles("ADMIN")
                        .build(),

                User.withUsername("user")
                        .password("{noop}user123")
                        .roles("USER")
                        .build()
        );
    }
}
```

Conclusion

The AI Tool Finder backend successfully demonstrates a clean, scalable, and functional system for managing AI tools and user reviews. Through well-designed entities, repositories, services, and controllers, the application allows:

- Users to discover AI tools with advanced filtering by category, pricing, and ratings.
- Users to submit reviews, which are stored with a pending status to ensure moderation.
- Admins to add, update, or delete tools and approve or reject reviews, maintaining data quality.
- Automatic average rating computation based on only approved reviews, ensuring accurate representation of tool quality.

By following a layered architecture, the backend separates business logic, data access, and request handling, making the system maintainable, extendable, and production ready. Overall, this project provides a practical simulation of a real-world backend platform that balances user functionality and admin control.